GESTIÓN DE BASES DE DATOS

Proyecto

SoundAI

Diseño del modelo entidad-relación y paso al modelo relacional

AUTORES

Nicole Sofía Díaz Carneiro

Derek Márquez Cruz

MÓDULO

Gestión de Bases de Datos

Índice de contenidos

01 Supuesto 3
1.1 Descripción general 3
1.2 Usuarios 3
1.3 Listas de reproducción 3
1.4 Asistente de chat 4
1.5 Catálogo musical 4
1.6 Reglas de la aplicación 5
02 Modelo entidad-relación 7
2.1 Diseño en CrowsFootModel 7
2.2 Relaciones del modelo 7
03 Paso al modelo relacional 9
3.1 Reglas de transformación 9
3.2 Modelo relacional 9
3.3 Esquema relacional 10
3.3.1 Usuarios 11
3.3.2 Listas de reproducción 11
3.3.3 Chat 11
3.3.4 Artistas 12
3.3.5 Canciones 12

PARTE 1

01 Supuesto

1.1 Descripción general

SoundAI es una plataforma de **música en línea simplificada**. Permite a sus usuarios organizar canciones en listas de reproducción y, en el caso de los usuarios de pago, mantener conversaciones con un asistente que les recomienda canciones. El catálogo musical está formado por **artistas, álbumes y canciones**. La base de datos debe almacenar toda esta información de forma consistente y sin redundancias.

1.2 Usuarios

Cada usuario se identifica mediante un **código único y se registra con un correo electrónico y un nombre de usuario**. Todo usuario es de uno de estos dos tipos, y nunca de ambos a la vez: **usuario gratuito o usuario premium.**

Del usuario gratuito se guarda el **número de anuncios que se le han reproducido y el límite de saltos de canción** que tiene permitidos. Del usuario premium se guarda la **fecha de renovación de su suscripción y el método de pago.**

1.3 Listas de reproducción

Un usuario puede **crear tantas listas de reproducción como quiera**, pero cada lista **pertenece** a un **único usuario**. De cada lista se guarda su **nombre, su fecha de creación** y si está **marcada como favorita.**

Una lista contiene elementos ordenados. Cada elemento ocupa una posición dentro de su lista y registra la **fecha en la que se añadió** y un comentario opcional. Además, hace referencia a una única canción, y una misma canción puede aparecer en muchas listas. Un elemento no tiene sentido fuera de su lista: se identifica por la **lista a la que pertenece** y por su **posición en ella.**

1.4 Asistente de chat

Solo los usuarios premium pueden **iniciar sesiones de chat** con el asistente. De cada sesión se **guarda un título y la fecha de inicio.**

Una sesión está formada por mensajes numerados dentro de ella. De cada mensaje se almacena el rol de quien lo emite (por ejemplo, usuario o asistente) y su texto. Un mensaje puede **recomendar varias canciones**, y una misma canción puede ser recomendada en muchos mensajes.

1.5 Catálogo musical

Cada artista se identifica mediante un código único y tiene un nombre artístico y un país. Todo artista es de uno de estos dos tipos, y nunca de **ambos a la vez**: solista o banda. De un solista se guardan su nombre real y su fecha de nacimiento. De una banda se guardan su año de formación y su nombre.

Un artista puede publicar varios álbumes, pero cada álbum lo publica un único artista. De cada álbum se guarda su **título y su año de lanzamiento.** Un álbum contiene **varias canciones**, y de cada canción se almacena su nombre y su duración en segundos. Una canción puede ser, además, una **versión (remix)** de otra canción del catálogo.

Finalmente, los artistas participan en las canciones mediante créditos. Cada crédito indica el rol que el artista desempeña en la canción, el nombre con el que se le acredita y si figura como artista invitado. Un mismo artista puede tener **varios roles en una misma canción.**

1.6 Reglas de la aplicación

- **Tipos de usuario:** Todo usuario es gratuito o premium, nunca ambos.

- **Chat reservado:** Únicamente los usuarios premium pueden usar las sesiones de chat

- **Propiedad de las listas:** Cada lista pertenece a un único usuario.

- **Orden de las listas:** Dentro de una lista, cada elemento ocupa una posición distinta.

- **Tipos de artista:** Todo artista es solista o banda, nunca ambos.

- **Autoría de los álbumes:** Cada álbum lo publica un único artista.

- **Versiones:** Una canción puede ser un remix de otra canción del catálogo.

- **Créditos:** Un artista puede tener varios roles en la misma canción.

PARTE 2

02 Modelo entidad-relación

En esta parte se presenta el diseño conceptual a partir del supuesto anterior.

2.1 Diseño en CrowsFootModel

> **NOTACIÓN:** Los atributos que empiezan por # son identificadores, los que empiezan por * son obligatorios y los que empiezan por ° son opcionales. El círculo con una x indica una jerarquía de especialización, y (d,t) significa que es disjunta y total.

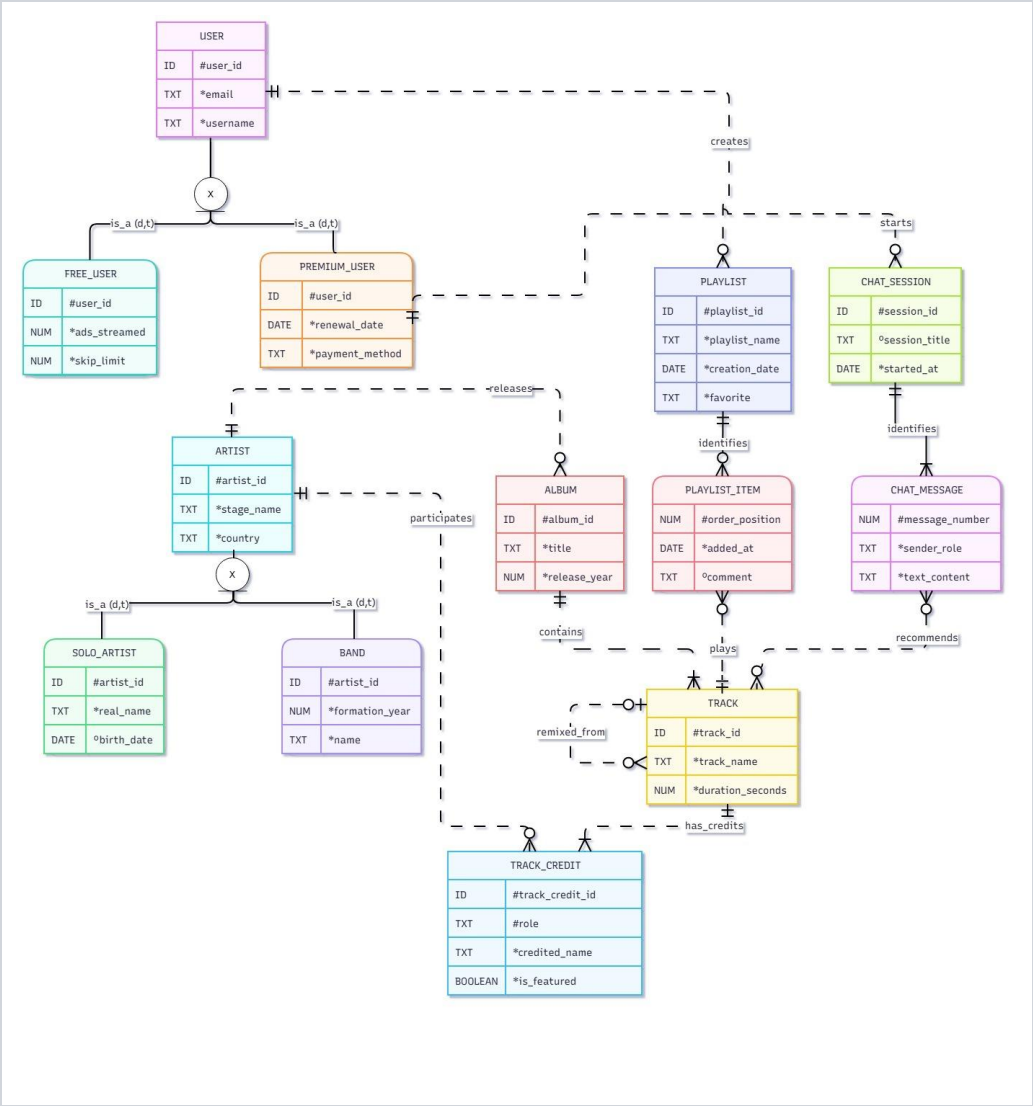


Figura 1. Modelo entidad-relación de SoundAI en CrowsFootModel.

2.2 Relaciones del modelo

La tabla 1 resume las relaciones del modelo, las entidades que conectan y su cardinalidad.

Relación	Entidades	Cardinalidad	Descripción
is_a (usuarios)	USERS → FREE_USER, PREMIUM_USER	Jerarquía(d,t)	Todo usuario es gratuito o premium, y de uno solo de los dos tipos.
is_a (artistas)	ARTIST → SOLO_ARTIST, BAND	Jerarquía(d,t)	Todo artista es solista o banda, y de uno solo de los dos tipos.
creates	USERS – PLAYLIST	1:N	Un usuario crea muchas listas; cada lista pertenece a un usuario.
identifies	PLAYLIST – PLAYLIST_ITEM	1:N	Los elementos se identifican por su lista y su posición.
plays	PLAYLIST_ITEM – TRACK	N:1	Cada elemento apunta a una canción, que puede estar en varias listas.
starts	PREMIUM_USER – CHAT_SESSION	1:N	Un usuario premium inicia varias sesiones de chat.
identifies	CHAT_SESSION - CHAT_MESSAGE	1:N	Los mensajes se identifican por su sesión y su número.
recommends	CHAT_MESSAGE – TRACK	N:M	Un mensaje recomienda varias canciones y una canción puede ser recomendada muchas veces.
releases	ARTIST – ALBUM	1:N	Un artista publica varios álbumes; cada álbum tiene un artista.
contains	ALBUM – TRACK	1:N	Un álbum contiene varias canciones.
remixed_from	TRACK – TRACK	1:N reflexiva	Una canción puede ser un remix de otra.
has_credits / participates	TRACK – ARTIST - (TRACK_CREDIT)	N:M con atributos	Participación con rol, nombre acreditado e indicador de invitado.

PARTE 3

03 Paso al modelo relacional

En esta parte se transforma el modelo entidad-relación en un modelo relacional y se detalla el esquema resultante.

3.1 Reglas de transformación

Para obtener el modelo relacional se han aplicado estas reglas de transformación:

1. **Entidades:** Cada entidad es una tabla y su identificador, la clave primaria.

2. **Jerarquías:** Cada subtipo es una tabla cuya clave primaria es también clave foránea hacia su super tipo (free_users, premium_users, solo_artists y bands).

3. **Relaciones 1:N:** La clave del lado 1 pasa como clave foránea a la tabla del lado N.

4. **Entidades débiles:** Su clave primaria incluye la clave de la entidad de la que dependen sumando su propio discriminante: playlist_items (playlist_id, order_position) y chat_messages (session_id, message_number).

5. **Relaciones N:M:** Se crea una tabla intermedia con las claves primarias de las entidades relacionadas. Como chats_recommends o tracks_credit, que ademas incluye el rol en la clave

6. **Relación reflexiva:** En tracks, remixed_from_id referencia a track_id y admite valores nulos, porque no todas las canciones son remixes.

3.2 Modelo relacional

La figura 2 muestra el modelo relacional con las claves primarias, claves foráneas de cada tabla y las relaciones entre ellas.

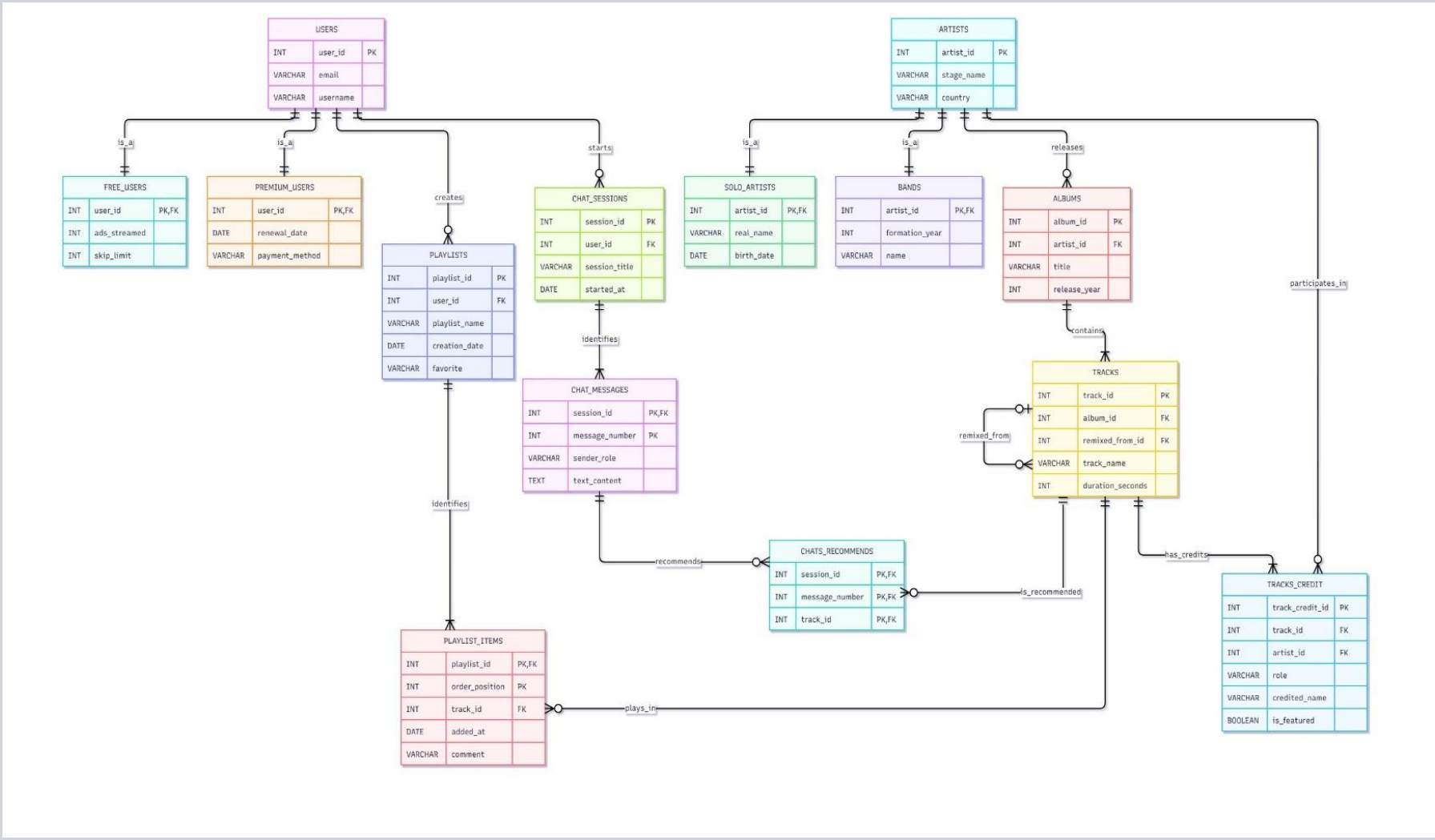


Figura 2. Modelo relacional de SoundAI.

3.3 Esquema relacional

Se detalla el esquema relacional en forma textual.

> **Notación:** **Las claves primarias (PK) aparecen subrayadas y en negrita, las claves foráneas (FK) en cursiva, y los atributos que son a la vez PK y FK en negrita, subrayado y cursiva.**

3.3.1 Usuarios

```
users (user_id, email, username)

free_users (user_id, ads_streamed, skip_limit)
WHERE {user_id} REFERENCES users (user_id)

premium_users (user_id, renewal_date, payment_method)
WHERE {user_id} REFERENCES users (user_id)
```

3.3.2 Listas de reproducción

```
playlists (playlist_id, user_id, playlist_name,
creation_date, favorite)
WHERE {user_id} REFERENCES users (user_id)

playlist_items (playlist_id, order_position, track_id,
added_at, comment)
WHERE {playlist_id} REFERENCES playlists (playlist_id)
WHERE {track_id} REFERENCES tracks (track_id)
```

3.3.3 Chat

```
chat_sessions (session_id, user_id, session_title,
started_at)
WHERE {user_id} REFERENCES users (user_id)

chat_messages (session_id, message_number, sender_role,
text_content)
WHERE {session_id} REFERENCES chat_sessions (session_id)

chats_recommends (session_id, message_number, track_id)
WHERE {session_id, message_number} REFERENCES chat_messages
(session_id, message_number)
WHERE {track_id} REFERENCES tracks (track_id)
```

3.3.4 Artistas

```
artists (artist_id, stage_name, country)

solo_artists (artist_id, real_name, birth_date)
WHERE {artist_id} REFERENCES artists (artist_id)

bands (artist_id, formation_year, name)
WHERE {artist_id} REFERENCES artists (artist_id)
```

3.3.5 Canciones

```
albums (album_id, artist_id, title, release_year)
WHERE {artist_id} REFERENCES artists (artist_id)

tracks (track_id, album_id, remixed_from_id, track_name,
duration_seconds)
WHERE {album_id} REFERENCES albums (album_id)
WHERE {remixed_from_id} REFERENCES tracks (track_id)

trackscredit (track_credit_id, track_id, artist_id, role,
credited_name, is_featured)
WHERE {track_id} REFERENCES tracks (track_id)
WHERE {artist_id} REFERENCES artists (artist_id)
```